

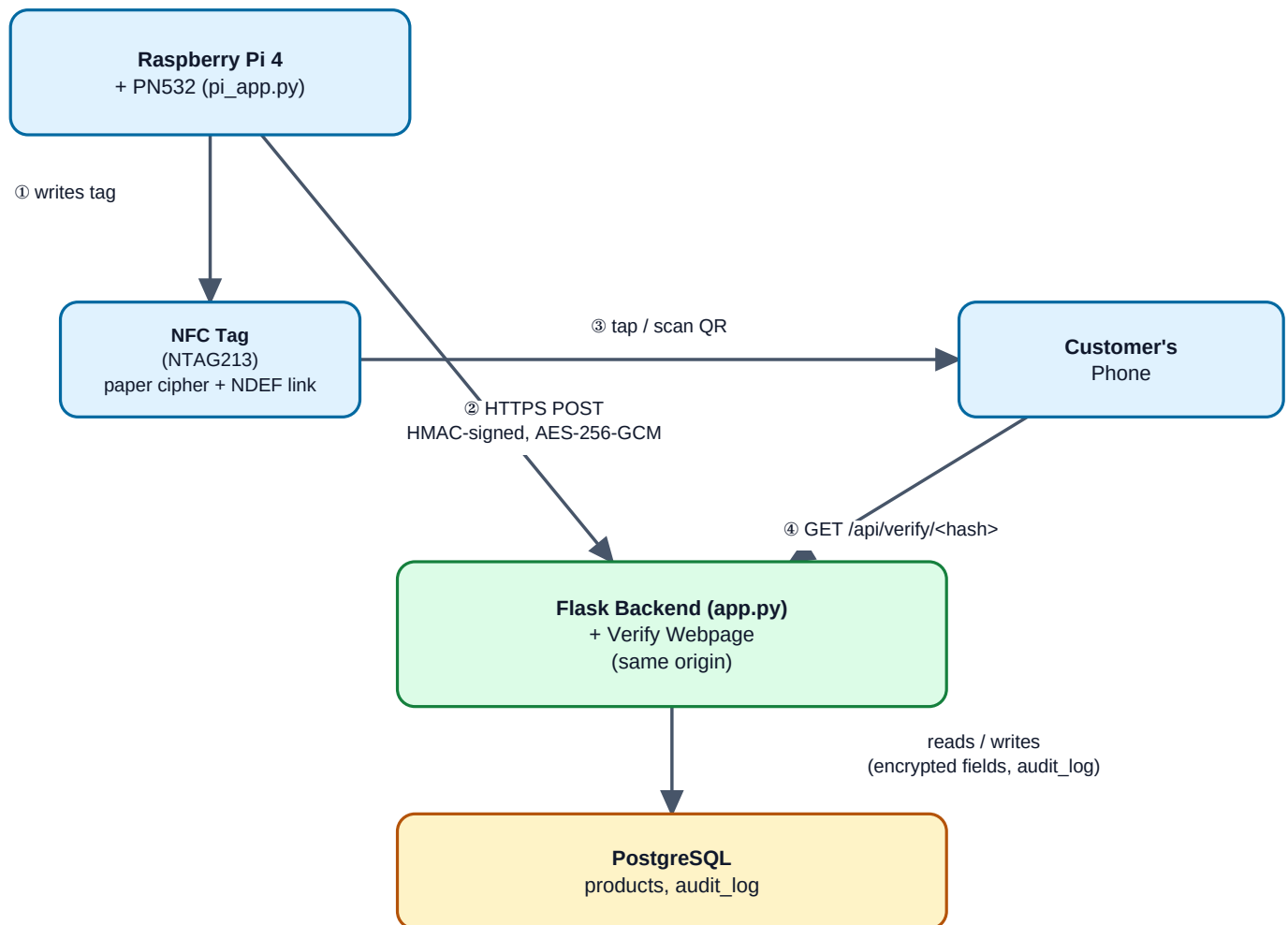
NFC Medicine Authenticity System

Architecture & Security Design • how it works, why it's secure, and what's still needed

This document explains how the system is put together, the specific security mechanisms in place and why each one exists, and the work still required before it can be treated as a finished, deployable system.

1. System Architecture

There are three physical/logical components: the **Raspberry Pi** (packaging-time writer), the **Flask/PostgreSQL backend** (single source of truth), and the **consumer's phone** (verification-time reader). The backend also serves the verification webpage itself, so the customer never talks to a separate frontend server.



Two independent flows share the same backend: the write flow (top-left, run once per product at packaging time) and the verify flow (top-right, run every time a customer checks a tag).

1.1 Write flow (packaging time)

- The Pi reads the tag's hardware UID and derives per-tag key material locally from it (HMAC/HKDF over the UID and a shared secret) — nothing is generated on the tag itself.
- It writes a 16-byte ciphertext block onto the tag using the custom “paper cipher” (pages 4–7), plus a standard NDEF URI record (pages 8+) that points a phone's browser at the verify webpage for that tag.
- It separately encrypts the full product record with AES-256-GCM and POSTs it to the backend over HTTP(S), signed with HMAC-SHA256 so the backend can confirm the request really came from a Pi that holds the shared secret.
- The backend validates the signature, checks the nonce hasn't been used before, decrypts only the manufacture date (to compute an expiry date), and stores everything else as ciphertext.

1.2 Verify flow (point of sale / point of use)

- Tapping the tag (or scanning its QR fallback) opens the verify webpage with the tag's identifier in the URL.
- The webpage calls GET /api/verify/<tag_uid_hash> on the backend.
- The backend looks up the record, decrypts it **server-side**, checks it for tampering and expiry, and returns a plain status: authentic, expired, tampered, or unknown.
- The phone's browser never receives ciphertext, keys, or any secret — only the final answer and, if authentic, the product's display details.

2. Security Design

Each mechanism below addresses a specific way the original localhost demo could have been abused. They're described as independent layers — defeating one doesn't defeat the others.

2.1 Write authentication (who is allowed to register a product)

Every `POST /api/products` request must carry an HMAC-SHA256 signature over the request timestamp and body, computed with a secret (`SHARED_SECRET`) that only the Pi and the backend know. Without it, the request is rejected (403) before any data is touched. This stops anyone from registering a counterfeit product record, even if they can reach the backend's network address.

2.2 Replay protection (stopping a captured request from being resent)

- A **30-second timestamp window** rejects requests signed too long ago.
- A **database-level UNIQUE constraint on the nonce column** rejects a reused nonce outright (409) — this holds even if an attacker could produce a fresh, currently-valid signature over the same nonce, because it's enforced independently of signature validity or timing.

2.3 Confidentiality (protecting stored data)

Two encryption paths exist side by side, tagged per-record by a `crypto_version` column:

- **paper_v1** — the original custom cipher described in the accompanying research paper. Kept byte-for-byte unchanged so the paper's results stay reproducible. Not intended as the system's real security boundary going forward.
- **aes_gcm_v1** — the production path. AES-256-GCM with a key derived per-tag via HKDF-SHA256 from a master secret. Critically, **no key material is ever transmitted in a request or stored in the database** — the Pi and the backend each derive the same key independently from the tag's hash and the shared master secret, so there's nothing in transit or at rest for an attacker to steal and reuse.

2.4 Integrity / tamper detection

- AES-GCM includes a built-in authentication tag, so any single-field ciphertext modification is caught automatically on decryption.
- A SHA-256 hash over all four encrypted fields together (`payload_hash`) is stored alongside the record and re-checked on every verify, catching cross-field tampering (e.g. copying one genuine ciphertext into a different record) that a per-field auth tag alone wouldn't notice.
- Either check failing flips the verify result straight to **tampered**, which takes priority over every other status.

2.5 Safe consumer verification

This is the architectural decision that makes a browser-based scan safe: decryption and the authenticity decision happen entirely on the server. The response contains only a status word and, if authentic, plain display text. There is no code path by which a customer's phone (or anyone inspecting its network traffic) can obtain a key, a secret, or raw ciphertext.

2.6 Access control

Bulk/administrative endpoints (`/api/admin/products`, `/api/admin/keys/<hash>`) require a static `X-Admin-Key` header, compared using a constant-time comparison to avoid timing side-channels. The consumer verify endpoint is intentionally left open — it has to be reachable by any customer's phone — but it never returns anything sensitive.

2.7 Rate limiting

Every route has a per-IP request limit (the verify endpoint, being the most exposed, is capped at 20/minute). This blunts both brute-force signature guessing against the write endpoint and mass enumeration of tag hashes against the verify endpoint.

2.8 Audit logging

Every write attempt and every verify attempt — successful or not — is recorded in an `audit_log` table with the event type, result, tag hash, source IP, and timestamp. This is both a monitoring tool and the evidence trail behind the attack-simulation test results.

3. Threat Model: What's Defended Against

Attack / scenario	Defense	Verified by
Replay a captured write request	Nonce UNIQUE constraint + 30s timestamp window	backend/tests/test_replay.py
Register a counterfeit product (no shared secret)	HMAC signature requirement on every write	backend/tests/test_clone.py
Scan a cloned/blank tag UID	Server-side lookup returns "unknown" for unregistered hashes	backend/tests/test_clone.py
Brute-force signatures / enumerate tag hashes	Rate limiting (429) on every route	backend/tests/test_bruteforce.py
Tamper with ciphertext in transit	Signature check runs before decryption; forged bodies never reach storage	backend/tests/test_mitm.py
Tamper with a stored record directly (e.g. compromised DB access)	payload_hash mismatch / GCM auth-tag failure → flagged "tampered"	backend/tests/test_mitm.py
Leak key material from network traffic or the database	Keys are derived independently, never transmitted or stored (aes_gcm_v1 path)	backend/crypto_modern.py

Each row above corresponds to a passing automated test — run `pytest tests/ -v` in `backend/` to reproduce these results directly, and `tests/report.py` to generate a written evidence report cross-checked against the live `audit_log` table.

4. Known Limitations (Documented, Not Solved)

- **Physical tag cloning:** the system verifies by server-side lookup, not by cryptographically proving the physical chip is genuine. A cloned tag broadcasting a copied UID and NDEF link would still resolve to the real record and show authentic. Closing this needs tag-side password protection (NTAG213 PWD_AUTH) or an originality-signature chip family.
- **Single shared-secret trust:** both crypto paths rely on one HMAC secret to authenticate that a write came from the real Pi. If it leaks, writes can be forged under either path. Ed25519 device-signing (private key never leaves the Pi) would remove this single point of failure.
- **Confidentiality without TLS:** the attack-simulation results prove tamper *detection* works even without HTTPS — they do not prove traffic is private. A LAN eavesdropper can still observe plaintext HTTP metadata today. This is exactly what Phase 2 (HTTPS deployment) addresses.

5. What's Still Needed

5.1 Required before this can be demoed or trusted

- Connect a real PostgreSQL database (none was reachable while this was built) and run `init_db.py` then `migrate.py`.
- Run the automated test suite end-to-end and generate the evidence report.
- Test the write and NDEF-write flow against real PN532 + NTAG213 hardware — the tag-writing logic has only been verified by pure byte-math so far, not a physical write.

5.2 Phase 2 — deployment

- Replace the Flask development server with a production WSGI server (waitress/gunicorn).
- Put the backend behind HTTPS (e.g. Cloudflare Tunnel) and update both `.env` files to the public URL.
- Revisit the rate limiter's storage backend if the server ever runs multiple worker processes.

5.3 Optional further hardening

- NTAG213 PWD_AUTH page-password protection, to stop the NDEF record on a genuine tag from being physically overwritten.
- Ed25519 device-signing on the Pi, replacing the single shared-secret trust model.

5.4 Research-paper extras

- Benchmarking Ed25519 vs. the current approach for the paper's performance numbers.
- Running the NIST SP 800-22 randomness test suite against the nonce generator.

Generated by Claude as an architecture and security reference for this project. See [NFC_Project_Changes_Report.pdf](#) for the detailed change log, and [git log](#) for full history.